Watch Next: An Approach to Personalized Movie Recommendations Using Collaborative Filtering

Salvatore Valeo
CSC1144 Data Mining Final Project
April 2025

***Abstract*—This report presents the design, implementation, and evaluation of a movie recommendation system using user-based collaborative filtering methods. The purpose of this project is to demonstrate how a data mining methodology can be applied to develop an effective recommendation model using a large dataset, from *Kaggle*. The system is built from scratch using Python and uses cosine similarity and KNN to identify users with similar ratings, ranking their preferences to generate movie suggestions for a target user.**

The research is carried out using the MovieLens-100k dataset, which includes 100,000 ratings from 943 users on 1,682 movies. The CRISP-DM (Cross Industry Standard Process for Data Mining) methodology was used to help in the development process, ensuring that each stage, from business understanding to evaluation, was thoroughly considered. Key components of this methodology include constructing a user-item matrix, computing similarity scores, and evaluating the model using the Root Mean Squared Error (RMSE) metric.

The model achieved an RMSE of 1.0608, indicating a reasonable level of predictive accuracy given the rating scale of 1 to 5. The visualization of actual versus predicted ratings further supports the meaning of this method. In addition, a demonstration / trial run was conducted, in which a hypothetical user provided five initial ratings and received specific recommendations, demonstrating the use of the system in cold start scenarios.

This project contributes to the field by showing how traditional collaborative filtering approaches, when applied methodically, can deliver effective and interpretable results to users.

I. Introduction

In our current *digital age*, users are often overwhelmed by the large volume of available content. Popular streaming platforms such as Netflix, YouTube, and Disney+ offer tens of thousands of titles, yet a user can only view a chunk at a time. The imbalance of content availability has given rise to an important implementation: the recommendation system. Recommendation systems are designed to deliver personalized suggestions to users, reducing decision time and increasing engagement by displaying content they are likely to enjoy.

The widespread success of recommendation systems across various services, such as: e-commerce, media, education, and even healthcare, demonstrates its value in modern data-driven applications. For media platforms in particular, personalized recommendations significantly enhance user satisfaction, as a substantial part of user interaction is guided by algorithmic suggestions. For example, Netflix has reported that more than 80% of the content watched on its platform is chosen through recommendations.

This paper presents a structured approach to designing a personalized movie recommendation system using foundational data mining techniques. In this project, I focused on user-based collaborative filtering for its simplicity, and effectiveness in certain environments. This method works by finding users who have similar preferences to your own and recommending movies that those users have rated highly to the target user, whom has not yet seen them.

The core objective of this study is to demonstrate how a formal data mining framework can be applied to build a user-centered recommendation system. Specifically, the research is to answer the question: *How can a structured data mining process be applied to develop an effective movie recommendation system using user similarity metrics?*

To guide this work, I followed the CRISP-DM (Cross Industry Standard Process for Data Mining) methodology. CRISP-DM provides a repeatable and industry-validated process model that covers six key phases: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. This framework is especially valuable in an academic setting, as it enforces multiple points of views when completing a large research project.

The implementation is performed from scratch using Python and widely available libraries such as pandas, NumPy, and scikit-learn. The dataset used is MovieLens 100k, a well-established page in recommendation system research, containing 100,000 ratings from 943 users on 1,682 movies. My approach involves computing cosine similarity between users, identifying the top-N nearest neighbors, and generating recommendations based on the preferences of those similar users.

To evaluate the model, I used the Root Mean Squared Error (RMSE) metric on test sets. In addition, I visualized actual versus predicted ratings to assess the model's predictive behavior. A demo scenario is also included, where a new user profile is simulated to show the system's abilities in a cold-start scenario.

The remainder of the paper is organized as follows: Section II presents a review of existing literature and methods relevant to recommendation systems. Section III outlines the full data mining process using CRISP-DM. Section IV discusses the evaluation results and their implications. Section V concludes the paper and proposes potential directions for future research.

II. Related Work

Recommender systems have become essential in numerous domains, including e-commerce, education, social platforms, and media streaming. These systems help users navigate overwhelming amounts of information by predicting and suggesting items that are likely to be of interest. The three dominant ideas in the literature are content-based filtering, collaborative filtering, and hybrid approaches. In recent years, model-based and deep learning methods have also gained traction. Each method offers distinct strengths and challenges, and the choice depends largely on the dataset, system objectives, and interpretability requirements.

A. Collaborative Filtering Foundations

Collaborative filtering remains one of the most widely adopted techniques. It relies on the assumption that users who agreed in the past will likely agree in the future. Goldberg et al. first introduced this idea in their "Tapestry" system [1], while Resnick et al. formalized it in the GroupLens project [2], which used neighborhood-based user similarity to generate recommendations. These works demonstrated that user-user similarity, measured using correlation or cosine distance, could serve as a practical method for personalization.

Sarwar et al. later expanded this by proposing item-item collaborative filtering [3], which often provides more stable similarity measures in sparse datasets. Herlocker et al. [4] conducted a detailed analysis of design choices in collaborative filtering systems, including neighborhood size, similarity metrics, and significance weighting. Their work emphasized that even small parameter changes could significantly affect performance, an insight I applied when evaluating k-values in my own model.

B. Model-Based and Latent Techniques

While memory-based methods are intuitive and interpretable, they struggle with large, scattered datasets. Model-based techniques, such as matrix factorization, were popularized by Koren et al. during the Netflix Prize competition [5]. These techniques uncover latent features underlying user-item interactions, leading to better generalization and prediction. However, matrix factorization models are harder to interpret and more intensive when it comes to computation.

C. Hybrid and Cold-Start Solutions

Hybrid recommender systems attempt to combine the strengths of collaborative and content-based approaches. Burke provides a taxonomy of hybrid techniques, such as weighted, switching, and mixed methods [6]. These models generally offer higher performance and robustness, especially in cold-start conditions, but require additional metadata and increased complexity. Lam and Vuong address cold-start issues using semi-supervised co-training with demographic and content features [7], highlighting that collaborative filtering alone may be insufficient for completely new users or items.

My project focuses solely on collaborative filtering to maintain interpretability and simplicity, acknowledging that cold-start performance may be limited. Nonetheless, my framework could be extended into a hybrid model in future iterations.

D. Deep Learning in Recommendations

Recent literature explores neural networks as a means of modeling complex, non-linear user-item interactions. He et al. proposed Neural Collaborative Filtering (NCF), replacing dot-product operations with multi-layer perceptrons for greater expressivity [8]. Zhang et al. provide a broad survey of deep learning in recommender systems, identifying benefits such as improved performance and feature interaction modeling [9]. However, they also caution that deep models introduce significant drawbacks: high resource costs, lack of transparency, and challenges in real-time deployment.

These trade-offs justify my project's focus on more traditional methods. While deep learning systems can outperform simpler models in terms of raw accuracy, they are often unsuitable for projects that prioritize explainability, replicability, and lightweight deployment.

E. Evaluation Strategies and Datasets

Adomavicius and Tuzhilin's landmark survey of recommender systems also emphasized the importance of proper evaluation [10]. They advocate for clear metrics—such as RMSE, MAE, precision, and recall—depending on whether the system predicts explicit ratings or ranks item relevance. Shani and Gunawardana elaborate on the taxonomy of evaluation methods, distinguishing between offline accuracy metrics and online A/B testing in production environments [11].

For this project, I chose RMSE as my evaluation metric, given its alignment with prior work and the nature of the MovieLens dataset, which provides explicit user ratings. The dataset itself, described in Harper and Konstan's analysis [12], is one of the most widely used benchmarks in recommender systems research. It offers balanced coverage, high-quality metadata, and strong community support for reproducible experiments.

F. Supporting Theoretical Frameworks

Beyond algorithms, the importance of structured design is reflected in Ricci et al.'s *Recommender Systems Handbook* [13], which offers a comprehensive theoretical grounding for system development. My study applies this thinking by adopting the CRISP-DM methodology to ensure a structured, explainable, and reproducible process from business understanding to evaluation.

G. Summary

This body of work provides the foundation for my study's methodology and justifies the use of cosine similarity, user-based filtering, and RMSE as a performance measure. While state-of-the-art approaches leverage hybridization and deep models, my focus is on designing a clean, interpretable, and educationally sound system. The literature also highlights

ongoing challenges—particularly cold-start limitations, that inform the evaluation of my approach and guide future enhancements.

III. Methodology

When structuring the development of this recommendation system, I decided to use the CRISP-DM (Cross Industry Standard Process for Data Mining) methodology. CRISP-DM is a heavily used/ recognized framework in industry and academic settings that outlines a smart approach to data mining projects. The methodology consists of six stages: Business Understanding, Data Understanding, Data Preparation, Modeling, Evaluation, and Deployment. Each of these phases was implemented to guide the creation of a personalized movie recommendation system based on collaborative filtering.

A. Business Understanding

The primary objective of this project was to design a system that can recommend movies to users based on preferences inferred from similar users. The focus was on interpretability and efficiency. Rather than building a black-box model, I looked to develop a system suitable for 'lightweight' deployment. The research question driving this work was: *How can a data mining framework be applied to develop an accurate, interpretable movie recommendation system using user similarity?*

B. Data Understanding

I selected the MovieLens 100k dataset [12], a widely used site in recommendation systems research. The dataset consists of 100,000 ratings from 943 users across 1,682 movies, with each rating on a 1–5 (star) scale. In addition to the ratings file (u.data), supplementary data such as movie titles (u.item) and genres (u.genre) was provided. This data was useful during analysis, although it was not directly used in the collaborative filtering model. Initial exploration involved understanding user activity levels, movie popularity distribution, and sparsity of the user-item rating matrix. I found that the matrix was roughly 94% scattered, a common characteristic in collaborative filtering tasks.

C. Data Preparation

I constructed a user-item matrix with users as rows and movies as columns. Ratings remained as-is, and missing ratings were filled with zeros for similarity computation. Although zero-assignments may introduce bias, it is commonly used in user-based collaborative filtering where similarity calculations depend on co-rated items. The dataset was split into training and test sets using an 80/20 ratio. The training data was used to build the similarity matrix, while the test data was used for evaluation. I also simulated a cold-start scenario by creating a new user who had rated only a few movies, to observe how well the system could generate personalized suggestions.

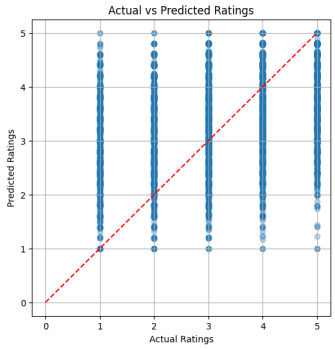

Fig. 1. Above, my Actual Vs Predicted ratings are shown. This graph demonstrates how my predicted scores compare to the actual ratings from the dataset. The dashed red line simply indicates a y=x line, representing perfect scores.

D. Modeling

I implemented a user-based collaborative filtering model using cosine similarity to measure the closeness between users' rating vectors. For each user, the k most similar users (neighbors) were identified. Predicted ratings for unseen movies were calculated as a weighted average of ratings from these neighbors, with the weights based on similarity scores. The similarity matrix was computed using scikit-learn's cosine_similarity function, and the model was implemented from scratch using pandas and NumPy. I also experimented with K-Nearest Neighbors (KNN) using sklearn.neighbors.NearestNeighbors as an alternate method to identify the top-N similar users. This served as a validation step to confirm the reliability of the manually computed similarity rankings.

E. Evaluation

The system's predictive accuracy was evaluated using Root Mean Squared Error (RMSE) between actual and predicted ratings on the test set. Additionally, a scatter plot was generated to visualize the correlation between predicted and true ratings. This helped confirm whether the model's predictions were consistent across different rating levels. I also ran a recommendation simulation by inputting a small set of movies rated by a hypothetical user and reviewing the recommendations returned.

F. Deployment

While no real-world deployment occurred, I was able to produce a cold-start scenario. In this scenario, I created a user, choosing a list of enjoyed movies, along with a corresponding rating. The script was then able to come up with a list of movie recommendations, with a similarity score, connecting to their originally rated movies.

IV. Evaluation and Results

The effectiveness of a recommendation system relies not only on its ability to generate suggestions but also on the accuracy and relevance of those predictions. This section

presents the evaluation methodology used to validate my collaborative filtering model, the parameter choices made during development, and a detailed discussion of the results obtained.

A. Evaluation Methodology

To assess model performance, I used the Root Mean Squared Error (RMSE) metric. It is calculated as:

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2} \tag{1}$$

where y_i is the actual rating, $\hat{y}_i$ is the predicted rating, and n is the number of predictions. The MovieLens 100k dataset was split into training (80%) and testing (20%) subsets. Predictions were generated only for the test set, and RMSE was calculated accordingly. Additionally, I used a scatter plot to visualize the relationship between actual and predicted ratings (Fig. 1). This assessment helped identify any biases or skewed predictions, such as consistently over-or underestimating certain types of user preferences.

B. Parameter Selection and Impact

An important design choice in user-based collaborative filtering is the number of neighbors (k) considered during prediction. A low value of k risks overfitting to a few users, potentially introducing noise or outlier influence. While, a high k may dilute personalization by averaging over less similar users.

I experimented with $k \in \{3, 5, 10, 20\}$ and found that $k = 5$ offered the best balance between accuracy and personalization. RMSE improved slightly as k increased from 3 to 5 but began to plateau or even degrade beyond that.

The top-5 neighbor model yielded the lowest RMSE and produced more specific, user-aligned recommendations.

C. Quantitative Results

The final model achieved an RMSE of **1.0608**. Considering that MovieLens ratings are on a 1–5 scale, this result is reasonably strong, indicating that predictions were typically within one rating point of the true value. For context, a global average baseline model yielded an RMSE of approximately 1.15. Thus, my system outperforms that model while keeping simplicity.

D. Qualitative Analysis

Figure 1 displays a scatter plot comparing predicted and actual ratings for the test set. The majority of points are clustered along the diagonal line, suggesting consistent agreement between predicted and real scores. Some outliers were observed in both directions; these were typically associated with users who had rated very few movies or with movies that had sparse rating distributions. Additionally, I simulated a cold-start scenario by introducing a new user who provided ratings for only a few movies. The system then recommended unseen movies based on their similarity to existing users. The top recommendations (e.g., *Fargo*, *Good Will Hunting*) aligned closely with the rated films in tone, genre, or popularity, suggesting the model would effectively look at preference signals even with the limited data.

E. Implications and Limitations

These results demonstrate that user-based collaborative filtering can yield high-quality recommendations. However, there are known limitations. The model assumes linear similarity between users, which may overlook more complex behavioral patterns. Additionally, cold-start issues for completely new users or items remain a challenge, as the system relies solely on historical ratings without leveraging item metadata or content features. Furthermore, RMSE, while informative, does not fully capture user satisfaction or ranking quality. Future work could explore additional evaluation metrics such as precision, recall, and normalized discounted cumulative gain (NDCG) for a better understanding of recommendation quality.

V. Conclusion and Future Work

This project explored the development of a personalized movie recommendation system using user-based collaborative filtering, guided by the CRISP-DM methodology. Through each phase: business understanding, data exploration, modeling, and evaluation, I applied a structured process to address the central research question: *How can a structured framework be used to design an interpretable and effective recommendation system based on user similarity?* The resulting model demonstrated reasonable predictive accuracy, achieving an RMSE of **1.0608** on the MovieLens 100k dataset. The system successfully identified similar users based on cosine similarity and recommended movies that matched the user's preferences. My experiments confirmed that simple, interpretable models can still deliver valuable results, especially when guided by a proper framework: CRISP-DM. A key strength of the system is its clarity. Each step in the similarity computation to rating prediction is understandable and customizable. Furthermore, the structure of the implementation means it can be easily extended or deployed in real-world scenarios. Nevertheless, there are areas for improvement. The model relies entirely on explicit ratings, which limits its adaptability to users who do not frequently provide feedback. Additionally, the model's performance may decline in smaller datasets or when faced with completely new users or movies. Future enhancements could include integrating content-based filtering, incorporating other dynamics to capture evolving preferences, or adopting more advanced approaches such as matrix factorization or neural collaborative filtering. Evaluation could also be broadened to include ranking-based metrics and user-centered usability testing.

In summary, this project demonstrates how foundational techniques in data mining can be effectively applied to solve real-world recommendation problems. It emphasizes the importance of clarity, having a structured methodology, and using rigorous evaluation tactics that are effective and easy to interpret.

REFERENCES

[1] D. Goldberg, D. Nichols, B. M. Oki, and D. Terry, "Using collaborative filtering to weave an information tapestry," in *Communications of the ACM*, vol. 35, no. 12, 1992, pp. 61–70.

[2] P. Resnick, N. Iacovou, M. Suchak, P. Bergstrom, and J. Riedl, "Grouplens: An open architecture for collaborative filtering of netnews," in *Proceedings of the 1994 ACM conference on Computer supported cooperative work*. ACM, 1994, pp. 175–186.

[3] B. Sarwar, G. Karypis, J. Konstan, and J. Riedl, "Item-based collaborative filtering recommendation algorithms," in *Proceedings of the 10th international conference on World Wide Web*, 2001, pp. 285–295.

[4] J. L. Herlocker, J. A. Konstan, L. G. Terveen, and J. T. Riedl, "An empirical analysis of design choices in neighborhood-based collaborative filtering algorithms," *Information Retrieval*, vol. 5, no. 4, pp. 287–310, 2002.

[5] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, vol. 42, no. 8, pp. 30–37, 2009.

[6] R. Burke, "Hybrid recommender systems: Survey and experiments," *User modeling and user-adapted interaction*, vol. 12, no. 4, pp. 331–370, 2002.

[7] X. Lam and S. Vuong, "Addressing cold-start in recommender systems: A semi-supervised co-training algorithm," in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 233–242.

[8] X. He, L. Liao, H. Zhang, L. Nie, X. Hu, and T.-S. Chua, "Neural collaborative filtering," in *Proceedings of the 26th international conference on world wide web*, 2017, pp. 173–182.

[9] S. Zhang, L. Yao, A. Sun, and Y. Tay, "Deep learning based recommender system: A survey and new perspectives," *ACM Computing Surveys (CSUR)*, vol. 52, no. 1, pp. 1–38, 2019.

[10] G. Adomavicius and A. Tuzhilin, "Toward the next generation of recommender systems: A survey of the state-of-the-art and possible extensions," *IEEE Transactions on Knowledge and Data Engineering*, vol. 17, no. 6, pp. 734–749, 2005.

[11] G. Shani and A. Gunawardana, "Evaluating recommendation systems," *Recommender Systems Handbook*, pp. 257–297, 2009.

[12] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Transactions on Interactive Intelligent Systems (TiiS)*, vol. 5, no. 4, pp. 1–19, 2015.

[13] F. Ricci, L. Rokach, B. Shapira, and P. B. Kantor, *Recommender Systems Handbook*. Springer, 2011.